accounts app — Developer Guide

Internal auth + user-management app for Kapil Enterprises Inventory ERP. Written for Django 5.0.1 + PostgreSQL. All auth endpoints are rate-limited.

Table of Contents

Why this app exists

Django's default `User` model uses `username` as the login field. This factory ERP uses **email** as the only identity, so we subclass `AbstractUser`, null out `username`, and set `USERNAME_FIELD = "email"`.

The app owns:

- Custom `User` model + `Skill` M2M
- All authentication views (OTP, password, Google OAuth)
- User management CRUD (create / edit / delete users) — Super Admin only
- All security primitives: OTP utils, cache-backed rate limiter, allauth adapter

File map

```
config/accounts/
├── models.py           # User model, UserManager, Skill
├── views.py            # All views — auth, user management, skill management
├── forms.py            # SignupForm, UserCreateForm, UserEditForm, SkillForm
├── urls.py             # URL patterns (app_name = "accounts")
├── utils.py            # OTP generation, hashing, session helpers, email send
├── throttle.py         # Cache-backed rate limiter (no external deps)
├── allauth_adapters.py # Restricts Google sign-in to pre-provisioned users only
├── decorators.py       # login_required_view (function-view decorator)
├── admin.py            # Django admin registration for User + Skill
├── apps.py             # AppConfig
├── tests.py            # 21 security tests (argon2, forms, throttle, lockout,
etc.)
└── migrations/         # DB schema history
```

Auth flows

OTP LOGIN (PRIMARY)

The main way employees log in. No password needed — just email + a 6-digit OTP sent to that email.

```
Browser                          LoginView                        VerifyOTPView
   |                                 |                                 |
   |--- POST /app/ (email) --------->|                                 |
   |                         rate-check (otp_send)                     |
   |                         email exists? yes → generate OTP          |
   |                         hash OTP → store in session               |
   |                         send_otp_email()                          |
   |<-- redirect /app/verify-otp/ ---|                                 |
   |                                                                   |
   |--- GET /app/verify-otp/ ------------------------------------->    |
   |<-- render otp.html (email prefilled) --------------------------   |
   |                                                                   |
   |--- POST /app/verify-otp/ (otp=123456) ---------------------->    |
   |                                                rate-check (otp_verify)
   |                                                check_otp_from_session()
   |                                                match → login(user)
   |                                                reset throttle counters
   |<-- redirect /app/home/ (→ inventory dashboard) ----------------   |
```

Anti-enumeration: If the email does NOT exist, `LoginView` still stores the email in the session and redirects to the OTP page — the browser cannot tell whether the email was found or not.

OTP security (utils.py):

- `secrets.randbelow()` — OS CSPRNG, not `random`
- SHA-256 hash stored in session — raw OTP never persisted
- `hmac.compare_digest()` for comparison — no timing side-channel
- 5-minute expiry, 3-attempt cap per OTP (session-layer), throttle (cache-layer)

PASSWORD LOGIN (FALLBACK)

`/app/login/password/` — extends Django's built-in `AuthenticationView`.

```
Browser                          PasswordLoginView
   |                                    |
   |--- POST (username=email, password) -->|
   |                           rate-check (pw_login) ← BEFORE Django auth
   |                           blocked? → error + 429-style message
   |                           not blocked → super().post() → Django auth
   |                           wrong password → form_invalid → log warning
   |                           correct → form_valid → reset_throttle → login
   |<-- redirect /app/home/ ---------|
```

Lockout: 5 wrong passwords per email in 15 min → 30-min block. The throttle check runs *before* `authenticate()` so we never compute an Argon2 hash for a blocked IP/email (denial-of-service hardening).

SIGNUP

Self-registration for new users (e.g., a new worker getting their own account). Two-step: fill form → verify email OTP → account created.

```
Browser               SignupView              SignupVerifyView
   |                      |                         |
   |--- POST /signup/ ---->|                         |
   |               validate form (email, password, confirm)
   |               run validate_password()
   |               sign password with SECRET_KEY (django.core.signing)
   |               store signed token in session (never plain text)
   |               generate OTP → hash → session
   |               send_otp_email()
   |<-- redirect /signup/verify/ --|                 |
   |                                                 |
   |--- POST /signup/verify/ (otp) ---------------->|
   |                                        rate-check (otp_verify)
   |                                        check_otp_from_session()
   |                                        signing.loads(token)
   |                                        User.objects.create_user()
   |                                        login(user)
   |<-- redirect /app/home/ -----------------|
```

IntegrityError guard: `SignupForm.clean_email` no longer rejects existing emails (anti-enumeration). If two tabs race to create the same email, the `IntegrityError` from the DB `UNIQUE` constraint is caught in `SignupVerifyView` and the user is quietly redirected to login with a neutral message.

FORGOT PASSWORD

```
Browser             ForgotPasswordView      ResetPasswordVerifyView
   |                      |                         |
   |--- POST (email) ---->|                         |
   |               rate-check (otp_send)            |
   |               email exists? send OTP           |
   |               ALWAYS: same message shown       |
   |<-- redirect /reset-password/verify/ --|        |
   |                                                |
   |--- POST (otp, new_password) ---------------------->|
   |                                        rate-check (otp_verify)
   |                                        check_otp_from_session()
   |                                        validate_password(new_password)
   |                                        user.set_password()
   |                                        reset ALL throttle counters
   |<-- redirect /login/password/ (email prefilled) ----|
```

Anti-enumeration: Same “If this email is registered, an OTP has been sent” message regardless of whether the email was found.

GOOGLE OAUTH

Handled by `django-allauth`. The custom adapter in `allauth_adapters.py` restricts it to pre-provisioned users only.

```
Browser               allauth (Google)          RestrictedSocialAccountAdapter
   |                        |                                |
   |--- click "Sign in with Google" -->|                     |
   |                        |-- Google callback --- pre_social_login() -->|
   |                                                  email in User DB?
   |                                                  NO  → redirect login + er-
ror
   |                                                  YES → connect social ac-
count
   |<-- logged in (or error) -------------------------------|
```

Why restricted? `SOCIALACCOUNT_AUTO_SIGNUP = True` (allauth default) lets ANY Google account create a new User row on first sign-in. For an internal-only factory ERP that is a serious access-control hole. The adapter inverts this: a Super Admin must create the User row first (via `/app/users/add/`), then the worker can use their Google account to sign in.

User model

```
accounts.User (AbstractUser)
├── email           EmailField, unique=True, USERNAME_FIELD
├── first_name      CharField
├── last_name       CharField
├── phone_number    CharField (optional, regex validated)
├── birth_date      DateField (optional)
├── bio             TextField (optional)
├── salary          DecimalField (optional)
├── profile_picture ImageField (optional)
├── user_type       CharField, choices: admin/manager/karigar/helper/normal
├── role            FK → inventory.Role  ← RBAC source of truth
├── skills          M2M → Skill
├── is_active       BooleanField
├── is_staff        BooleanField
└── is_superuser    BooleanField
```

`user_type` vs `role`:

- `user_type` = legacy display label (shows in UI chips, used for team overview)

- `role` = RBAC source of truth (what the user can DO — which views they can access, what sidebar items appear). Always use `permission_service.user_has_role()` / `user_has_perm()` in views, never check `user_type` for access control.

RBAC integration

`views.py` imports from `inventory.services`:

```
from inventory.services import user_has_role, ROLE_SUPER_ADMIN
```

`SuperuserRequiredMixin` gates all user-management and skill-management views:

```
class SuperuserRequiredMixin(UserPassesTestMixin):
    def test_func(self):
        return user_has_role(self.request.user, {ROLE_SUPER_ADMIN})
```

Rule (CLAUDE.md #6): Never use `is_superuser` directly in views. Always go through `permission_service`. This keeps the permission logic in one place.

Rate limiting

`throttle.py` — cache-backed, no external dependencies.

Two axes per request:

Axis	What it tracks	Why
`ip`	Originating IP address	Blocks mass attempts from one host
`id`	Email / username	Blocks credential-stuffing across rotating IPs

Per-email limits are tighter than per-IP because the factory office NATs ~30 workers behind one IP. Punishing the IP would lock out the whole floor.

Four scopes:

Scope	Used by	Per-email limit	Block duration
`otp_send`	LoginView, ResendOTPView, ForgotPasswordView	5 OTPs / 10 min	15 min
`otp_verify`	VerifyOTPView, SignupVerifyView, ResetPasswordVerifyView	10 attempts / 10 min	30 min
`pw_login`	PasswordLoginView	5 wrong passwords / 15 min	30 min
`signup`	SignupView	3 attempts / 15 min	15 min

Usage pattern:

```
# At the top of a POST handler:
blocked, retry = check_throttle(request, "pw_login", identifier=email)
if blocked:
    messages.error(request, f"Try again in {format_retry(retry)}.")
    return error_response

# On successful auth:
reset_throttle("pw_login", identifier=email, request=request)
```

Production note: Default `LocMemCache` resets per gunicorn worker process. Swap to Redis (`django_redis` or `django.core.cache.backends.redis`) so counters are shared across all workers.

Self-protection guards

Prevent a Super Admin from accidentally locking themselves — and everyone else — out of the system.

`UserUpdateView.form_valid` — when a Super Admin edits their own profile:

- Cannot revoke their own `is_superuser` flag
- Cannot deactivate their own account (`is_active = False`)
- Cannot change their own RBAC role away from `super_admin`

`UserDeleteView.form_valid`:

- Cannot delete their own account
- Cannot delete another Super Admin if they are the only remaining active one (would leave the platform with zero admins — shell access required to recover)

Security decisions log

Decision	Reason
Argon2id as default hasher	OWASP recommendation; resistant to GPU/ASIC cracking. PBKDF2 kept as fallback — Django auto-upgrades hashes on next login.
`validate_password()` on all forms	Catches short, numeric-only, and common passwords before they reach the DB.
SHA-256 OTP hash in session	Plain OTP never persisted; timing-safe comparison via `hmac.compare_digest`.
`signing.dumps` for signup password	Avoids plain-text password in session between signup step 1 and step 2.
`SESSION_COOKIE_HTTPONLY = True`	JS cannot read session cookie — XSS protection.
`SESSION_COOKIE_SAMESITE = "Lax"`	Blocks cross-site cookie send — CSRF mitigation layer.
DB-backed sessions (`SESSION_ENGINE = "...backends.db"`)	Sessions are server-revocable; cookie-only sessions cannot be invalidated.
`SOCIALACCOUNT_AUTO_SIGNUP = False`	Prevents random Google accounts from entering the ERP.
Audit log (`accounts.security` logger → `logs/security.log`)	Flat `key=value` pairs for grep / Loki / Splunk. All auth success + failure events captured.

Adding a new auth endpoint

1. Add the view to `views.py` with throttle checks:

```
def post(self, request):
    email = request.POST.get("email", "").strip().lower()

    # Always check throttle before any heavy work.
    blocked, retry = check_throttle(request, "otp_send", identifier=email)
    if blocked:
        messages.error(request, f"Try again in {format_retry(retry)}.")
        return render(request, "accounts/your_template.html")

    # ... your logic ...

    # On success, reset counters.
    reset_throttle("otp_send", identifier=email, request=request)
    security_logger.info("your_event_name email=%s", email)
```

2. If it's a new throttle scope, add a row to `LIMITS` in `throttle.py`:

```
"your_scope": {
    "ip": Limit(hits=20, window=600, block=900),
    "id": Limit(hits=5,  window=600, block=900),  # keep id tighter than ip
},
```

3. Add a URL to `urls.py`.
4. Write tests in `tests.py` — at minimum: throttle blocks after N hits, resets on success.

Running tests

```
# From the repo root:
cd config
../env/bin/python manage.py test accounts --settings=config.settings.local
```

Expected: **21 tests, 0 failures.**

Test coverage:

- Argon2 hashing (`PasswordHashingTests`)
- Signup / create / edit form validation (`SignupFormTests` , `UserCreateFormTests` , `UserEditFormTests`)
- Self-protection (`SelfProtectionTests`)
- Per-IP and per-email throttle + reset (`ThrottleTests`)

- Anti-enumeration on login (`AntiEnumerationTests`)
- Password login lockout + reset on success (`PasswordLoginLockoutTests`)

All test classes inherit `BaseSecurityTest` which pins the cache backend to `LocMemCache` and calls `cache.clear()` in setUp/tearDown to keep counters isolated between test methods.

How data flows through this app (added 2026-06-12)

```
login (password/OTP/Google) → rate-limit check (cache, per IP+email)
  → session → every request: permission_service.user_has_perm/_has_role
  → SidebarItemRule (menu + URL gated together, via inventory middleware)
```

Tables: `accounts_user` (custom User, email login) · `accounts_role` + `accounts_user_extra_roles` (RBAC; relocated here 2026-06 so identity+access is ONE foundation) · `accounts_sidebaritemrule` · allauth tables (Google).

Who writes: user CRUD → accounts services (service layer owns writes, 2026-06 remediation removed the signals); role/sidebar edits → Access hub views via the same services. **Single-writer note:** `user_has_perm` carries the `ROLE_SUPER_ADMIN` bypass — change it nowhere else.

Why this design / what breaks if bypassed: auth is the foundation gate — a raw `is_superuser` check in a view bypasses the role editor's curation and breaks the delegation model (rule 6). Skipping the rate-limiter helpers on a new auth endpoint reopens brute-force.

Django Learning Notes

- **Custom User (`AUTH_USER_MODEL`)**: email as username; set BEFORE first migrate, effectively permanent.
- **Argon2 hasher** + cache-backed rate limit (per IP+email) on every auth endpoint; allauth restricted to pre-provisioned users (no open signup).
- **M2M `extra_roles`**: one primary FK role + additive extras — simple precedence, no role explosion.